Programmation orientée objet

Classes, objets et les trois piliers

BTS CIEL — 1re année | Python

Plan

L'idée de la POO

Classe et objet

Le constructeur

Les trois piliers

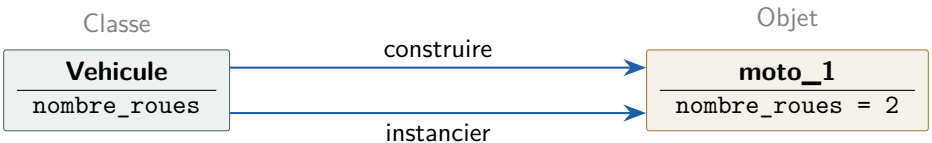
Compléments

Qu'est-ce que la POO ? L'analogie

Idée principale : programmer en décrivant des **objets**, à l'image de ceux de la vie réelle, plutôt qu'une simple suite d'instructions.

L'analogie de la voiture

- on écrit une *recette* (un modèle) qui décrit ce qu'est une « voiture » ;
- à partir de cette recette, on fabrique autant de voitures concrètes qu'on veut.



La « recette » → la classe

> **Classe** : modèle (ou plan, ou moule) qui définit les **caractéristiques** et les **comportements** des objets créés à partir d'elle.

Une classe définit deux choses :

- les **attributs** — ce qu'un objet *a* couleur, marque, vitesse
- les **méthodes** — ce qu'un objet *fait* `rouler()`, `freiner()`

```python
class Voiture:
    # Le constructeur : initialise les attributs
    def __init__(self, couleur, marque):
        self.couleur = couleur
        self.marque  = marque
        self.vitesse = 0

    # Une méthode (action)
    def rouler(self):
        self.vitesse += 10
```

L'objet → l'instance

Objet : exemplaire concret créé à partir d'une classe. On dit que c'est une **instance** de cette classe.

Un objet peut représenter une entité du monde physique ou un concept : un individu, une voiture, un nombre complexe, une matrice...

```
# Instanciation : on crée deux objets Voiture
ma_voiture = Voiture("rouge", "Ferrari")
ta_voiture = Voiture("bleue", "Renault")

ma_voiture.rouler()   # La Ferrari roule à 10 km/h
ta_voiture.rouler()   # La Renault roule à 10 km/h
```

Différence clé : `Voiture` est le *plan* ; `ma_voiture` est une voiture *réelle et unique*. Chaque objet est indépendant, mais suit les règles de sa classe.

Le constructeur `__init__`

> **Constructeur** : méthode spéciale appelée **automatiquement** à la création (instanciation) d'un objet. Elle initialise les attributs et prépare l'état initial de l'objet.

```python
class Voiture:
    def __init__(self, marque, modele, annee, couleur="blanc"):
        # Initialisation des attributs d'instance
        self.marque       = marque
        self.modele       = modele
        self.annee        = annee
        self.couleur      = couleur
        self.kilometrage  = 0        # Valeur par défaut
        self.est_demarree = False
```

Trois façons d'appeler le constructeur

```
# Création avec tous les paramètres
voiture1 = Voiture("Toyota", "Corolla", 2020, "rouge")

# Création avec paramètre optionnel par défaut
voiture2 = Voiture("Peugeot", "208", 2019)

# Création avec paramètres nommés
voiture3 = Voiture(marque="BMW", modele="X3", annee=2022, couleur="noir")
```

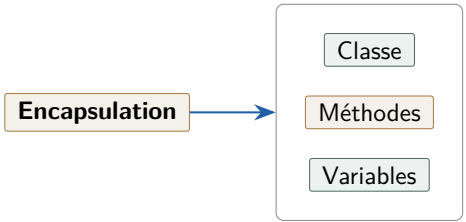
À retenir

`self` désigne l'objet en cours de construction : il est passé automatiquement par Python, on ne le fournit jamais à l'appel.

Pilier 1 — l'encapsulation

Principe

- on « cache » les détails internes ;
- on protège les données en contrôlant leur accès ;
- on évite les modifications accidentelles et on garantit la cohérence des données.



Attributs et méthodes sont soit **publics** (accessibles de l'extérieur), soit **privés** (accessibles seulement via des méthodes dédiées).

> **En Python, par convention** : `solde` public — `_solde` protégé (usage interne) — `__solde` privé (accès via `deposer()`, `retirer()`).

> **Héritage** : une classe « enfant » hérite des attributs et méthodes d'une classe « parent ».

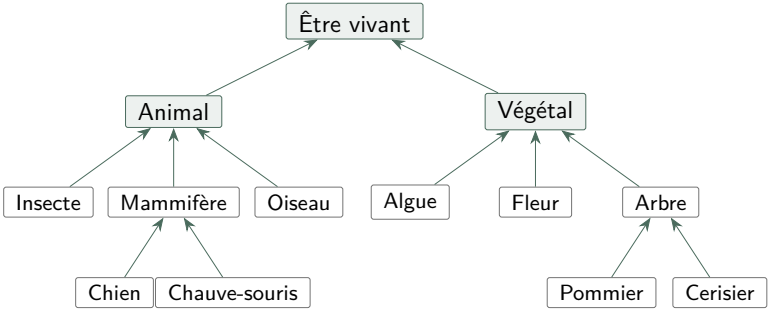
Avantages

- réutilisation du code : rien à réécrire ;
- spécialisation : une `Voiture` *est un* `Vehicule` ;
- l'enfant peut ajouter ses propres méthodes ou **surcharger** celles du parent.

```
class Vehicule:                  # Parent
    def se_deplacer(self):
        print("Le véhicule se déplace.")

class Voiture(Vehicule):         # Enfant
    def se_deplacer(self):       # Surcharge
        print("La voiture roule.")
```

Héritage : une hiérarchie de classes



Lecture

la flèche se lit « *est un* » : un `Pommier` est un `Arbre`, donc un `Végétal` — et il hérite de tout ce qu'elles définissent.

Pilier 3 — le polymorphisme

Polymorphisme : « plusieurs formes » : une même interface (une méthode de même nom) pour des objets de types différents. Chaque objet réagit à sa manière.

```
class Voiture(Vehicule):
    def se_deplacer(self):
        print("La voiture roule.")

class Bateau(Vehicule):
    def se_deplacer(self):
        print("Le bateau navigue.")

for v in [Voiture(), Bateau()]:   # Même appel, comportements différents
    v.se_deplacer()
```

Intérêt

le code appelant ignore le type exact : ajouter un `Avion` ne change rien à la boucle.

Attributs de classe et attributs d'instance

Attribut d'instance

- propre à chaque objet ;
- défini avec `self.nom_attribut` ;
- ex. le titulaire d'un compte.

Attribut de classe

- partagé par tous les objets ;
- une seule copie en mémoire ;
- ex. le nom de la banque.

```
class Compte:
    banque = "Banque Nationale"   # Attribut de
     classe

    def __init__(self, titulaire):
        self.titulaire = titulaire   # Attribut
     d'instance
```

modifier `Compte.banque` change la valeur *pour tous* les comptes.

Accesseurs et mutateurs (*getters* & *setters*)

> **Rôle** : donner un accès **contrôlé** à un attribut privé : l'accesseur (*getter*) le lit, le mutateur (*setter*) le modifie après vérification.

```
class Compte:
    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire
        self.__solde   = solde          # Attribut privé

    def get_solde(self):                # Accesseur
        return self.__solde

    def deposer(self, montant):         # Mutateur contrôlé
        if montant <= 0:
            raise ValueError("Le montant doit être positif.")
        self.__solde += montant
```

La version pythonique : `@property`

```
class Compte:
    def __init__(self, solde=0):
        self.__solde = solde

    @property
    def solde(self):                # Lecture : compte.solde
        return self.__solde

    @solde.setter
    def solde(self, valeur):        # Écriture : compte.solde = 100
        if valeur < 0:
            raise ValueError("Le solde ne peut pas être négatif.")
        self.__solde = valeur
```

Avantage

on écrit `compte.solde` comme un attribut ordinaire, mais le contrôle de cohérence reste assuré par la classe.

Bilan

Notion	Définition en une ligne
Classe	le modèle : attributs + méthodes
Objet	une instance concrète de la classe
Constructeur	`__init__`, appelé à la création, initialise l'état
Encapsulation	cacher l'interne, contrôler l'accès aux données
Héritage	une classe enfant réutilise et spécialise un parent
Polymorphisme	un même appel, des comportements différents

Suite : TP POO Python — écrire ses propres classes, encapsuler les attributs, construire une hiérarchie d'héritage.